

第09章_传输信道处理_上_学习笔记

传输链路上半部分学习总结

本文总结了前面讨论的“传输信道处理”上半部分内容，重点包括：链路整体流程、速率匹配方法、收发缓冲区设计与示例、为什么速率匹配往往优先包含更多系统比特、加扰的数学原理，以及 DFT-s-OFDM 的链路特性与其对多流传输的限制。

1. 传输链路的整体流程

从基带比特到最终发射波形，可以把整个链路粗略写成：

传输块 TB → TB CRC → 码块分段（可选） → CB CRC（可选） → LDPC 编码 → 速率匹配 / HARQ → 加扰 → 调制

其中每一步的作用如下：

1.1 传输块与 CRC

上层交给物理层的是一个 **传输块 (Transport Block, TB)**。为了检测最终解码是否正确，先给整个传输块加上 **TB CRC**。

如果传输块太大，超过了单个 LDPC 码块允许的长度，就需要把 TB 切分成若干 **码块 (Code Block, CB)**。每个码块还可以再加一个 **CB CRC**，便于分块校验、分块重传和分块译码。

因此，CRC 的核心作用是：

- **错误检测**，不是主要的纠错手段；
- TB CRC 负责整块检测；
- CB CRC 负责小块检测，便于后续 HARQ / 分段处理。

1.2 LDPC 编码

NR 数据信道主要采用 **LDPC 编码**。输入是“信息比特 + CRC 比特”，输出是更长的编码码字：

$$c = [b, p]$$

其中：

- **b：系统比特 (systematic bits)**，本质上是输入比特本身；
- **p：奇偶校验比特 (parity bits)**，由 LDPC 编码器生成；

- 总输出码字长度大于输入长度，因此引入了冗余，以支持纠错。

这里要特别区分：

- **CRC 比特**：在 LDPC 之前加，是被保护对象的一部分；
- **LDPC parity bits**：由 LDPC 编码器生成，用来支持纠错；
- parity bits 不是“又做了一次新的编码”，而是 **LDPC 编码本身的输出组成部分**。

1.3 速率匹配与 HARQ

LDPC 编码之后得到的是一个固定结构的较长码字，但一次实际 PDSCH/PUSCH 传输能承载的比特数取决于：

- 分配了多少 PRB；
- 持续了多少 OFDM 符号；
- 扣掉了多少 DM-RS / PT-RS / 控制开销；
- 是否还有预留资源需要绕开。

因此，编码后码字的长度通常**不等于**这次真正可以发的比特数。此时就需要：

- **速率匹配**：从编码比特中挑出“这次要发的那一部分”；
- **HARQ**：若本次译码失败，下次重传时再挑另一部分，接收端再把多次结果联合起来利用。

1.4 加扰

速率匹配之后，对输出比特序列做加扰：

$$\tilde{b}(n) = b(n) \oplus c(n)$$

其中：

- $b(n)$ ：速率匹配后的比特；
- $c(n)$ ：伪随机扰码序列；
- $\oplus$ ：模 2 加法 / 异或。

它的主要作用不是增加纠错能力，而是让待调制比特更“随机化”，并让不同用户/不同小区的信号在错误解扰后更像噪声。

1.5 调制

加扰后的比特被映射为调制符号，例如：

- QPSK
- 16QAM
- 64QAM
- 256QAM
- 在 DFT-s-OFDM 上行场景下还可支持 $\pi/2$ -BPSK

1.6 OFDM / DFT-s-OFDM 发射

- **普通 OFDM**：调制符号直接映射到各个子载波，再做 IFFT；
- **DFT-s-OFDM**：在映射到子载波之前，先对调制符号做一次 DFT，再进行子载波映射和 IFFT，因此整体发射波形更接近单载波。

2. 速率匹配的基本思想

2.1 为什么需要速率匹配

假设 LDPC 编码输出长度为 N ，而本次物理资源可承载比特数为 E 。通常有：

$$E \neq N$$

于是必须从长度为 N 的编码比特中选出长度为 E 的一部分，作为本次真正送往调制器的比特序列。

因此，速率匹配可以理解成：

让“编码器生成的比特数”与“本次实际可发的比特数”匹配。

2.2 速率匹配的核心步骤

从逻辑上看，速率匹配可以概括为三步：

第一步：组织编码比特

LDPC 输出的码字由系统比特和奇偶校验比特组成。标准通常会先把一部分固定位置的系统比特打孔，然后把剩余编码比特按既定顺序组织起来。

第二步：写入环形缓冲区

剩余编码比特被写入一个 **环形缓冲区 (circular buffer)**。写入顺序通常是：

1. 未打孔的系统比特；
2. 奇偶校验比特。

第三步：从不同起点读出

根据：

- 这次需要发送的比特数 E ；
- HARQ 使用的冗余版本 RV；

从环形缓冲区里读取一段连续比特。若读到尾部不够，就绕回开头继续读。

因此：

- **发送多少比特**：由资源量决定；
- **从哪里开始读**：由 RV 决定。

3. 发送端和接收端缓冲区的设计

“缓冲区”这个词在这一部分里其实有两种含义：

1. **发端环形缓冲区**：用于速率匹配；
2. **收端软缓冲区**：用于 HARQ 软合并。

3.1 发端环形缓冲区

发端环形缓冲区的本质是：

把 LDPC 编码后的比特组织成一个可循环读取的比特池。

其核心用途有三点：

(1) 支持速率匹配

本次需要发送多少比特，就从环形缓冲区中读取多少比特。

(2) 支持不同 RV

不同的 RV 对应不同的起始位置，因此同一个编码码字可以在不同重传中发送不同部分的冗余。

(3) 支持重复发送

若本次需要发送的比特数超过缓冲区中一圈可提供的有效独特比特数，则读完一圈后还可以从头继续读，这等价于某些比特被重复发送。

3.2 为什么叫“环形”

因为缓冲区首尾相连。读取过程不是简单的“到末尾就停止”，而是：

$N - 1 \rightarrow 0 \rightarrow 1 \rightarrow \dots$

这使得：

- 处理不同长度的发送需求更自然；
- RV 切换更方便；
- 支持 repetition 更方便。

3.3 收端软缓冲区

接收端软缓冲区是 HARQ 机制的关键。

它存储的不是简单的 0/1 判决，而是每个接收比特的**软信息**，例如 LLR。其本质是：

把前几次接收中得到的“带置信度的观测”先保存下来，以便后续重传到来时联合使用。

其作用包括：

- 保留第一次失败传输中的软信息；
- 在后续重传到来时，与新接收软信息做软合并；
- 提高译码成功率。

3.4 有限软缓冲区

接收端软缓冲区不是无限大的。实际上：

- HARQ 进程可能不止一个；
- 每个进程可能对应多个码块；
- 每个码块都需要存软信息；
- 高阶调制、大块数据会显著增加缓存需求。

因此，实际系统中终端缓存有限，这会限制 HARQ 可达到的性能，也会影响速率匹配和可支持的重传方式。

4. 速率匹配与 HARQ 的关系

4.1 HARQ 的本质

HARQ (Hybrid ARQ) 可以理解为：

- 一次传输失败后，系统允许重传；
- 重传时不是简单机械地“完全重复上一包”，而是可发送不同冗余；
- 接收端对多次接收结果进行联合利用。

4.2 为什么速率匹配和 HARQ 总放在一起讲

因为 HARQ 的“不同冗余版本”就是通过速率匹配来实现的。

同一个 LDPC 码字通过不同 RV、不同起点读取，可以得到不同的输出比特集合：

RV0, RV1, RV2, RV3

这些不同 RV 并不是携带不同的信息，而是：

对同一个编码码字，从环形缓冲区的不同位置开始取不同的一段。

4.3 接收端为什么要做软合并

假设第 1 次传输失败，第 2 次重传使用了新的 RV。接收端保留第 1 次的软信息，再将第 2 次的软信息叠加起来。

HARQ 的收益主要体现在两方面：

(1) 能量合并增益

若同一比特在多次传输中都被接收到，则接收端对它的观测更可靠。

(2) 编码增益

若不同 RV 发送了新的奇偶校验比特，则相当于接收端拿到了更多冗余信息，等效可用码率下降，更容易译码成功。

5. 一个速率匹配与缓冲区的小例子

假设某个 LDPC 码块编码后得到：

$$N = 3000 \text{ bits}$$

而这次 PDSCH 扣除 DM-RS 和其他开销后，只能承载：

$$E = 1200 \text{ bits}$$

则流程如下：

1. 将 3000 个编码比特按规则写入环形缓冲区；
2. 第一次传输使用 RV0，从起始点 k_0 开始读取 1200 bit；
3. 接收端尝试译码，若 CRC 失败，则发送 NACK；
4. 第二次传输改用 RV1，从另一个起始点 k_1 开始再读 1200 bit；
5. 接收端把两次软信息合并后重新译码；
6. 若仍失败，可继续使用 RV2 / RV3。

因此，这里每次传输发的都不是“全包重发”，而是：

从同一个大码字中抽出不同的一部分，让接收端逐步积累对该码字的观测。

6. 为什么速率匹配常倾向于选择更多系统比特

这个问题的关键在于：**NR 的 LDPC 是系统码 (systematic code)**。

因此编码输出可以写成：

$$\mathbf{c} = [\mathbf{b}, \mathbf{p}]$$

其中：

- **b**：系统比特，直接对应输入比特（信息 + CRC）；
- **p**：奇偶校验比特。

6.1 为什么系统比特更多时，初次解码更有利

接收端第一次收到的那一部分码字，若包含较多系统比特，就意味着译码器直接获得了更多“原始信息比特的带噪观测”。

从直观上讲：

- 系统比特：相当于直接观测原变量；
- parity bits：更多是提供约束关系。

若第一次传输里系统比特比较多，则：

- 译码器更容易获得足够多的“锚点”；
- 再配合一部分 parity bits 提供约束；
- 单次自解码的概率更高。

6.2 但这不等于“只发系统比特最好”

不是的。若只有系统比特而几乎没有 parity bits，则纠错能力会很弱。

更准确的说法是：

一个有利于初次解码的速率匹配子集，往往包含较多系统比特，同时又保留一定数量的 parity bits。

因此某些 RV（例如初始传输更常用的版本）之所以“更利于解码”，并不是因为它只包含系统比特，而是因为它更有可能选中“有利于单次译码”的那一部分码字结构。

7. 加扰的数学原理

7.1 最基本公式

加扰在比特域上的表达是：

$$\tilde{b}(n) = b(n) \oplus c(n)$$

其中：

- $b(n)$ ：待发送比特；
- $c(n)$ ：伪随机扰码序列；
- $\tilde{b}(n)$ ：加扰后的比特。

如果写成模 2 加法，就是：

$$\tilde{b}(n) = (b(n) + c(n)) \bmod 2$$

7.2 为什么有时又写成乘法

若把比特映射成双极性表示：

$$x(n) = 1 - 2b(n) \in \{+1, -1\}$$

扰码也写成双极性：

$$d(n) = 1 - 2c(n) \in \{+1, -1\}$$

则加扰后的结果可写为：

$$\tilde{x}(n) = x(n)d(n)$$

也就是说：

- 在 0/1 比特域里，加扰是异或；
- 在 ± 1 双极性域里，加扰是乘以 ± 1 。

两种写法本质等价。

7.3 接收端为什么能恢复

因为再次使用同一个扰码序列即可：

$$\hat{b}(n) = \tilde{b}(n) \oplus c(n) = b(n)$$

或在双极性域里：

$$\hat{x}(n) = \tilde{x}(n)d(n) = x(n)$$

因此加扰不是“加密”，而是一种可逆的伪随机化。

7.4 加扰的主要作用

(1) 让比特分布更随机

若直接发送高度规则的比特序列，例如长串 0 或长串 1，会导致调制符号统计特性不够好。加扰后，输出比特更接近均匀随机。

(2) 让不同用户/小区的信号可区分

不同链路采用不同初始化的扰码序列。若接收机用自己的扰码去解扰别人的信号，则别人的信号经过错误解扰后会进一步被随机化。

(3) 让干扰更像噪声

设本用户使用扰码 $c_u(n)$ ，干扰用户使用扰码 $c_i(n)$ 。对干扰信号做“错误解扰”时得到：

$$b_i(n) \oplus c_i(n) \oplus c_u(n)$$

由于 $c_i(n) \neq c_u(n)$ ，该结果仍然看起来是伪随机的，因此残余干扰更像噪声，而不是可利用的结构性模式。

(4) 帮助信道编码发挥作用

LDPC 等信道编码更擅长处理接近随机的误差，而不太希望干扰总是以某种固定模式集中打坏某些比特。加扰有助于将干扰和误差“去结构化”。

8. DFT-s-OFDM 的链路特性

8.1 普通 OFDM 与 DFT-s-OFDM 的区别

对普通 OFDM，若第 k 个子载波上的调制符号为 d_k ，则时域信号为：

$$x[n] = \frac{1}{\sqrt{N}} \sum_{k \in \mathcal{K}} d_k e^{j2\pi kn/N}$$

这里每个子载波上的符号是“相互独立映射”的。

对 DFT-s-OFDM，先对时域调制符号块 $d[m]$ 做 M 点 DFT：

$$D[\ell] = \sum_{m=0}^{M-1} d[m] e^{-j2\pi m\ell/M}$$

再把 $D[\ell]$ 映射到分配的子载波上并做 IFFT。于是每个发射样值不再只对应一个“本地子载波符号”，而是和整个符号块相关。

8.2 为什么说它更像单载波

DFT 预编码的作用可以理解为：

先把一串调制符号“扩散”到整个分配带宽上，再统一发出去。

结果是：

- 波形包络起伏更小；
- 峰均比（PAPR）更低；
- 更适合 UE 这种功放受限、功耗受限的上行发射端。

因此 DFT-s-OFDM 的主要优点是：

- 更低 PAPR；
- 更高功放效率；
- 更好的覆盖；
- 与 $\pi/2$ -BPSK 等低包络调制更匹配。

8.3 对上行链路设计的影响

由于 DFT-s-OFDM 先做 DFT，因此资源大小 M 本身会影响实现复杂度。为了简化实现，标准通常限制 DFT 长度对应的资源分配形态，这使得：

- 频域资源分配灵活性不如普通 OFDM；
- 不是任意 RB 大小都方便支持；
- 参考信号设计也需兼顾低 PAPR。

9. 为什么 DFT-s-OFDM 不适合灵活多流

更准确地说，不是“数学上绝对不能多流”，而是：

DFT-s-OFDM 与“高度频率选择性的多层空间预编码”天然存在张力。

9.1 多流与单载波之间的矛盾

设第 l 层的数据序列为 $d_l[m]$ ，先做 DFT 得：

$$D_l[k] = \sum_{m=0}^{M-1} d_l[m] e^{-j2\pi mk/M}$$

把所有层堆成向量：

$$\mathbf{D}[k] = \begin{bmatrix} D_1[k] \\ D_2[k] \\ \vdots \\ D_L[k] \end{bmatrix}$$

若再对每个子载波做空间预编码：

$$\mathbf{x}[k] = \mathbf{W}[k]\mathbf{D}[k]$$

接收端均衡后可写成：

$$\widehat{\mathbf{D}}[k] = \mathbf{A}[k]\mathbf{D}[k] + \hat{\mathbf{n}}[k]$$

其中 $\mathbf{A}[k]$ 是频点 k 上的整体等效变换。

接着做 IDFT 回到符号域：

$$\widehat{\mathbf{d}}[m] = \sum_{q=0}^{M-1} \mathbf{a}[m-q]\mathbf{d}[q] + \widehat{\mathbf{n}}[m]$$

其中：

$$\mathbf{a}[m-q] = \frac{1}{M} \sum_{k=0}^{M-1} \mathbf{A}[k] e^{j2\pi(m-q)k/M}$$

这个表达式非常关键。

9.2 若频域处理几乎不变，则单载波结构保留

若：

$$\mathbf{A}[k] \approx \mathbf{A}_0$$

即整个带宽上等效空间变换近似不变，则：

$$\widehat{\mathbf{d}}[m] \approx \mathbf{A}_0 \mathbf{d}[m]$$

此时每个符号主要只和自己有关，没有明显跨符号串扰，因此单载波结构基本还在。

9.3 若每个子载波都自由选不同预编码，则单载波结构被破坏

若 $\mathbf{W}[k]$ 随 k 剧烈变化，则 $\mathbf{A}[k]$ 也会强烈随频率变化，于是：

$$\widehat{\mathbf{d}}[m] = \mathbf{a}[0]\mathbf{d}[m] + \sum_{q \neq m} \mathbf{a}[m-q]\mathbf{d}[q] + \widehat{\mathbf{n}}[m]$$

这说明：

- 第 m 个输出符号不仅和自己有关；
- 还会混入别的 $q \neq m$ 的符号；
- 若是多层，还会伴随跨层耦合。

这就把原本想要的“单载波式整齐链路”变成了：

频率选择性空间处理 + 时域卷积 + 跨层耦合

从而：

- 接收机复杂度明显上升；
- 单载波优势被削弱；
- PAPR 也更容易被破坏。

9.4 更直观的理解

单载波体制希望的是：

整个带宽上的处理尽量统一、平滑。

而灵活多流 OFDM 往往希望：

每个子载波都根据自己的信道，单独选择最优预编码矩阵。

两者的偏好正好相反：

- 单载波：偏向宽带一致；
- 多流频域预编码：偏向逐子载波最优。

因此 DFT-s-OFDM 通常更适合：

- 单层；
- 或低层数、较受限的预编码；
- 不适合像普通下行 CP-OFDM 那样做高度自由的逐子载波多层空间复用。

10. 最后总结

可以把本部分内容压缩为下面几句话：

1. **传输链路流程**是：TB → CRC → 码块分段 → LDPC → 速率匹配/HARQ → 加扰 → 调制 → （上行可选）DFT 预编码 → 发射。
2. **速率匹配**的本质是：从编码后的大码字中挑出与本次物理资源长度匹配的一段；它通过环形缓冲区和不同 RV 同时支持 HARQ。
3. **发端环形缓冲区**负责组织编码比特并支持不同 RV；**收端软缓冲区**负责保存历史软信息并支持软合并。
4. **更有利于初次解码的速率匹配**通常会包含较多系统比特，因为系统比特提供了对原始信息的直接观测。
5. **加扰**的数学形式是模 2 加法（或双极性域中的符号翻转），其主要作用是随机化比特、区分链路、让干扰更像噪声。
6. **DFT-s-OFDM** 通过在映射子载波前增加 DFT，使上行波形更接近单载波、PAPR 更低，更适合 UE 发射。

7. **DFT-s-OFDM 不适合高度自由的多流预编码**，因为逐子载波变化的空间处理会破坏其单载波结构，并导致跨符号、跨层耦合。